Drugi popravni test iz Algoritama i struktura podataka

Prvi praktični test održan 19.02.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaA_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Insertion Sort*

Napisati ***Insertion Sort*** algoritam za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***InsertionSort*** napisati implementaciju za metodu ***public static void sort***(long[] arr)
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – *Proizvod elemenata između minimalne vrednosti i poslednjeg čvora*

Implementirati metodu koja pronalazi **prvo pojavljivanje minimalne vrednosti** u listi i vraća proizvod svih elemenata od tog čvora do kraja liste.

Dodatni zahtevi za rešavanje zadatka:

- Ako minimalna vrednost ne postoji (prazna lista), vratiti 0.
- Algoritam pisati u okviru klase ***LinkedList.***
- Algoritam mora imati vremensku kompleksnost **O(n)**.
- Zahtevan potpis metode: ***public int productFromFirstMin()***

Primer:

- Input: 4 → 2 → 3 → 4 ;
- Output: 24 (2 * 3 * 4)

- Input: 2 → 3 → 4 → 4 ;
- Output: 96 (2 * 3 * 4 * 4)

Zadatak 3 – *Fuzija brojeva (The Number Fusion)*

Zamisli da imaš niz brojeva koji ulaze u eksperimentalni reaktor jedan po jedan. Svaki broj predstavlja energetski nivo čestice. Kada nova čestica padne na onu ispod nje, pokreće se proces fuzije prema sledećim pravilima:

- **Fuzija (Spajanje):** Ako je nova čestica **veća ili jednaka** čestici na vrhu, one se spajaju u jednu novu česticu čija je vrednost jednaka njihovom **zbiru**. Novonastala čestica zatim pokušava da se spoji sa sledećom česticom ispod nje po istom pravilu.
- **Odbijanje:** Ako je nova čestica (ili novonastali zbir) **strogo manja** od čestice na vrhu, fuzija nije moguća i ona jednostavno ostaje da leži na njoj.
- **Prazan reaktor:** Ako je reaktor prazan, čestica samo zauzima mesto na dnu.

Tvoj cilj: Implementirati metodu koja prima niz vrednosti čestica i vraća finalno stanje u reaktoru (od dna ka vrhu).

<u>Dodatni zahtevi za rešavanje zadatka:</u>

1. Vremenska složenost: **O(n)**.
2. Rezultat mora biti niz u originalnom redosledu (od čestice na dnu do one na vrhu).

<u>Primer:</u>

Input 1 (nesortirani niz celih brojeva): [3, 2, 5]

Output 1: [10]

(5 se spaja sa 2 u 7; 7 se spaja sa 3 u 10)

Input 2 (nesortirani niz celih brojeva): [10, 2, 1, 4]

Output 2: [10, 7]

(*4 se spaja sa 1 u 5; 5 se spaja sa 2 u 7; 7 je manje od 10 i ostaje*)